

Prática 3 - Herança e Polimorfismo

Professor: Glender Brás

Exercício - Agenda Telefônica

Você foi contratado para implementar um aplicativo de agenda eletrônica. O sistema deverá permitir o cadastro e gerenciamento de diferentes tipos de contatos. Se necessário, você pode criar mais atributos e métodos do que os solicitados, desde que façam sentido para o contexto do problema.

Não se esqueça de criar os métodos `get` e `set` para todos os atributos.

1. Crie uma classe `Contato` com os seguintes atributos: `nome (String)`, `email (String)`, `telefone (String)`
2. Crie um método para exibir os dados do contato.
3. Crie as seguintes subclasses para `Contato`:
 - `ContatoPessoal` - Representa amigos e familiares. Atributos adicionais: `dataAniversario (String)`, `parentesco (String)`
 - `ContatoProfissional` - Representa contatos relacionados ao trabalho. Atributos adicionais: `empresa (String)`, `cargo (String)`
 - `ContatoEmergencia` - Representa contatos que podem ser acionados em situações de emergência. Atributos adicionais: `grauPrioridade (int - valor de 1 a 5)`, `observacao (String)`
4. Crie outra classe, chamada `Agenda`, que possui uma lista de contatos e um atributo para armazenar a quantidade de contatos cadastrados.
5. Na classe `Agenda` crie os seguintes métodos:
 - `Adicionar contato` - adiciona um contato à lista e atualiza o tamanho da lista.
 - `Remover contato` - excluir um contato da lista e atualiza o tamanho da Agenda. Retorne ao usuário se a exclusão foi bem sucedida ou não (contato encontrado ou não).
 - `Buscar contato por nome` - busca um contato pelo nome e retorna o contato encontrado (instância da classe `Contato`).
 - `Buscar contato por email` - busca um contato pelo email e retorna o contato encontrado (instância da classe `Contato`).
 - `Buscar contato por telefone` - busca um contato pelo nome e retorna o contato encontrado (instância da classe `Contato`).
6. Na classe `main` apresente um menu para o usuário com as seguintes opções:
 - (a) Adicionar contato
 - (b) Remover contato - caso o usuário não seja encontrado o usuário deve ser informado
 - (c) Buscar contato por nome - caso o contato não seja encontrado o usuário deve ser informado
 - (d) Buscar contato por email - caso o contato não seja encontrado o usuário deve ser informado
 - (e) Buscar contato por telefone - caso o contato não seja encontrado o usuário deve ser informado
 - (f) Consultar tamanho da Agenda - caso o contato não seja encontrado o usuário deve ser informado
 - (g) Finalizar - o programa só deve encerrar quando o usuário selecionar esta opção.

Exercício - Sistema de Venda de Ingressos para Eventos

ocê foi contratado para desenvolver um sistema de venda de ingressos para eventos. O sistema deverá permitir o cadastro de diferentes tipos de ingressos e o cálculo do valor final de cada ingresso de acordo com suas regras específicas.

1. Crie uma classe abstrata chamada Ingresso:
 - atributos: `codigo` (int), `nomeEvento` (String), `setor` (String), `valorBase` (double)
 - método concreto para exibir as informações básicas do ingresso
 - métodos abstratos: `double calcularValorFinal()`, `String obterBeneficios()`
2. Crie as seguintes subclasses para Ingresso:
 - IngressoComum - Representa um ingresso tradicional, sem descontos ou benefícios especiais.
 - Atributos específicos: Nenhum atributo adicional.
 - Regras:
 - * O valor final é igual ao valor base.
 - * Não possui benefícios adicionais.
 - IngressoEstudante - Representa ingressos adquiridos por estudantes.
 - Atributos específicos: `instituicaoEnsino` (String)
 - Regras:
 - * Possui 50% de desconto sobre o valor base.
 - * Deve informar como benefício o direito à meia-entrada.
 - IngressoVIP - Representa ingressos com acesso privilegiado ao evento.
 - Atributos específicos: `acessoBackstage` (boolean), `numeroLounge` (int)
 - Regras:
 - * Acréscimo de 80% sobre o valor base.
 - * Deve informar os benefícios disponíveis.
 - * Caso possua acesso ao backstage, esta informação deve ser exibida.
3. Crie uma chamada Bilheteria, que possui uma lista de ingressos.
4. Na classe Bilheteria crie os seguintes métodos:
 - Adicionar ingresso - Adiciona um ingresso à lista.
 - Remover ingresso - Remove um ingresso a partir do código informado. Retorne ao usuário se a operação foi realizada com sucesso.
 - Buscar ingresso - Recebe um código e retorna o ingresso correspondente.
 - Listar ingressos - Exibe todos os ingressos cadastrados. Utilize polimorfismo para que o sistema exiba corretamente as informações de cada tipo de ingresso.
 - Calcular arrecadação total - Percorre todos os ingressos cadastrados e retorna a soma dos valores finais dos ingressos. **Observação:** utilize o método `calcularValorFinal()` para realizar o cálculo.
 - Listar benefícios dos ingressos: Exibe os benefícios de todos os ingressos cadastrados. **Observação:** utilize o método `obterBeneficios()`.
5. Na classe *main* apresente um menu para o usuário com as seguintes opções:
 - (a) Cadastrar ingresso - o usuário deve escolher o tipo de ingresso.
 - (b) Remover ingresso - Caso o código não seja encontrado, o usuário deverá ser informado.
 - (c) Buscar ingresso por código - Exibir todas as informações do ingresso encontrado. Caso o código não exista, informar o usuário.
 - (d) Listar todos os ingressos - Exibir todos os ingressos cadastrados.
 - (e) Calcular arrecadação total - Exibir a soma dos valores finais de todos os ingressos cadastrados.
 - (f) Listar benefícios dos ingressos - Exibir os benefícios oferecidos por cada ingresso.
 - (g) Finalizar - o programa só deve encerrar quando o usuário selecionar esta opção.